Interesting though this is, it's a lot of effort to go to if all you get is a fancy way of computing some array bounds or an integral constant. The key benefit of constant expressions and `constexpr` functions involving user-defined types is that objects of a literal type initialized with a constant expression are statically initialized, and so their initialization is free from race conditions and initialization order issues:

```
CX si=half_double(CX(42,19));          <-- Statically initialized
```

This covers constructors too. If the constructor is declared `constexpr` and the constructor parameters are constant expressions, the initialization is *constant initialization* and happens as part of the static initialization phase. *This is one of the most important changes in C++11 as far as concurrency goes:* by allowing user-defined constructors that can still undergo static initialization, you can avoid any race conditions over their initialization, because they're guaranteed to be initialized before any code is run.

This is particularly relevant for things like `std::mutex` (see section 3.2.1) or `std::atomic<>` (see section 5.2.6) where you might want to use a global instance to synchronize access to other variables and avoid race conditions in *that* access. This wouldn't be possible if the constructor of the mutex was subject to race conditions, so the default constructor of `std::mutex` is declared `constexpr` to ensure that mutex initialization is always done as part of the static initialization phase.

A.4.2. constexpr objects

So far we've looked at `constexpr` as applied to functions. `constexpr` can also be applied to objects. This is primarily for diagnostic purposes; it verifies that the object is initialized with a constant expression, `constexpr` constructor, or aggregate initializer made of constant expressions. It also declares the object as `const`:

```
constexpr int i=45;                          <-- OK
constexpr std::string s("hello");            <-- Error, std::string isn't a literal type
int foo();
constexpr int j=foo();                       <-- Error, foo() isn't declared constexpr
```